

✓ Sentiment Analysis using Recurrent Neural Networks (LSTM)

Course: Deep Learning | University Project

Project Overview

This notebook implements a **complete Sentiment Analysis pipeline from scratch** using PyTorch. We classify tweets from the **Sentiment140** dataset (1.6 M tweets) as **positive** or **negative**.

What is built from scratch?

Component	Method
LSTM cell	Raw tensor ops — <code>(torch.matmul, torch.sigmoid, torch.tanh)</code>
Tokenizer	Whitespace split after regex cleaning
Vocabulary	Word-frequency counting — no pretrained tokenizer
Padding	Manual right-padding to fixed length

Dataset

Sentiment140 — 1.6 million English tweets Labels: 0 = Negative | 4 = Positive (remapped to 0 / 1) Source: Kaggle — `kazanov/sentiment140`

✓ Table of Contents

1. [Install & Import Libraries](#)
2. [GPU Setup & Random Seeds](#)
3. [Part A — Custom LSTM Theory](#)
4. [CustomLSTMCell Implementation](#)
5. [Comparison with PyTorch nn.LSTMCell](#)
6. [Part B — Load Dataset](#)
7. [Data Cleaning](#)
8. [Tokenization & Vocabulary](#)
9. [Data Splitting — 60 / 20 / 20](#)
10. [PyTorch Dataset & DataLoader](#)
11. [Part C — Model Architecture](#)
12. [Hyperparameters, Loss & Optimizer](#)
13. [Training & Validation Loop](#)
14. [Part D — Visualization](#)
15. [Part E — Test Evaluation](#)
16. [Conclusion & References](#)

```
# Install Kaggle API (needed to download the dataset in Colab)
!pip install -q kaggle
print('Installation complete!')
```

Installation complete!

```
# — Standard library —————
import os
import re
import math
import time
from collections import Counter

# — Data manipulation —————
import numpy as np
import pandas as pd

# — Visualisation —————
import matplotlib.pyplot as plt
import seaborn as sns

# — Machine-learning utilities —————
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report

# — PyTorch —————
```

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

print('All libraries imported successfully!')
print(f'PyTorch : {torch.__version__}')
print(f'Pandas : {pd.__version__}')
print(f'NumPy : {np.__version__}')
```

```
All libraries imported successfully!
PyTorch : 2.10.0+cu128
Pandas : 2.2.2
NumPy : 2.0.2
```

```
# — Reproducibility —————
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# — Device selection —————
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device : {device}')

if device.type == 'cuda':
    print(f'GPU : {torch.cuda.get_device_name(0)}')
    vram = torch.cuda.get_device_properties(0).total_memory / 1e9
    print(f'VRAM : {vram:.1f} GB')
else:
    print('No GPU found - training will be slow.')
    print('Tip: Runtime > Change runtime type > T4 GPU')
```

```
Device : cuda
GPU : Tesla T4
VRAM : 15.6 GB
```

Part A — Custom LSTM Theory

Why do vanilla RNNs fail on long sequences?

A simple RNN computes: $\mathbf{h}_t = \tanh(\mathbf{W} \cdot \mathbf{x}_t + \mathbf{U} \cdot \mathbf{h}_{t-1} + \mathbf{b})$

During backpropagation, gradients are multiplied by $\mathbf{W}$ at every step.

- If $|\mathbf{W}| < 1 \rightarrow$ gradients **vanish** $\rightarrow$ early tokens are forgotten
- If $|\mathbf{W}| > 1 \rightarrow$ gradients **explode** $\rightarrow$ training diverges

LSTM fixes this with a **cell state $\mathbf{C}_t$** updated by addition, letting gradients flow back hundreds of steps unchanged.

Official PyTorch LSTM Equations

At each time step t , given input $\mathbf{x}_t$, previous hidden state $\mathbf{h}_{t-1}$, previous cell state $\mathbf{C}_{t-1}$:

```
i_t = σ( W_ii·x_t + b_ii + W_hi·h_{t-1} + b_hi )  ← Input gate
f_t = σ( W_if·x_t + b_if + W_hf·h_{t-1} + b_hf )  ← Forget gate
g_t = tanh( W_ig·x_t + b_ig + W_hg·h_{t-1} + b_hg ) ← Candidate cell
o_t = σ( W_io·x_t + b_io + W_ho·h_{t-1} + b_ho )  ← Output gate

C_t = f_t ⊙ C_{t-1} + i_t ⊙ g_t                    ← Cell state update
h_t = o_t ⊙ tanh(C_t)                             ← New hidden state
```

Gate functions

Gate	Activation	Output range	Role
Forget $\mathbf{f}_t$	sigmoid	(0, 1)	0 = erase, 1 = keep old memory
Input $\mathbf{i}_t$	sigmoid	(0, 1)	how much new info to write
Candidate $\mathbf{g}_t$	tanh	(-1, 1)	what new content to write
Output $\mathbf{o}_t$	sigmoid	(0, 1)	which memory units to expose

Key insight: $\mathbf{C}_t$ is updated with **addition** ($\mathbf{f}_t \cdot \mathbf{C}_{t-1} + \mathbf{i}_t \cdot \mathbf{g}_t$), not $\tanh(\mathbf{W} \cdot \mathbf{C})$. Gradients flow straight back through $\mathbf{C}_t \rightarrow$ **no vanishing gradient**.

```
# =====
# CustomLSTMCell
# Implements the official PyTorch LSTM equations from scratch.
```

```

# Uses ONLY: torch.matmul torch.sigmoid torch.tanh
# Does NOT use nn.LSTMCell or nn.LSTM internally.
# =====

class CustomLSTMCell(nn.Module):
    """
    One LSTM time step computed from raw tensor operations.

    Args:
        input_size : number of features in input vector x_t
        hidden_size : number of features in hidden state h_t

    Call signature:
        h_t, c_t = cell(x_t, h_prev, c_prev)

    Tensor shapes:
        x_t [B, I] → h_t [B, H]
        h_prev [B, H] → c_t [B, H]
        c_prev [B, H]
        where B=batch, I=input_size, H=hidden_size
    """

    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size

        # — Input gate weights —————
        # W_ii maps x_t → hidden shape [H, I] (transposed in matmul)
        # W_hi maps h_{t-1} → hidden shape [H, H]
        self.W_ii = nn.Parameter(torch.empty(hidden_size, input_size))
        self.W_hi = nn.Parameter(torch.empty(hidden_size, hidden_size))
        self.b_ii = nn.Parameter(torch.zeros(hidden_size))
        self.b_hi = nn.Parameter(torch.zeros(hidden_size))

        # — Forget gate weights —————
        self.W_if = nn.Parameter(torch.empty(hidden_size, input_size))
        self.W_hf = nn.Parameter(torch.empty(hidden_size, hidden_size))
        self.b_if = nn.Parameter(torch.zeros(hidden_size))
        self.b_hf = nn.Parameter(torch.zeros(hidden_size))

        # — Candidate cell state weights —————
        self.W_ig = nn.Parameter(torch.empty(hidden_size, input_size))
        self.W_hg = nn.Parameter(torch.empty(hidden_size, hidden_size))
        self.b_ig = nn.Parameter(torch.zeros(hidden_size))
        self.b_hg = nn.Parameter(torch.zeros(hidden_size))

        # — Output gate weights —————
        self.W_io = nn.Parameter(torch.empty(hidden_size, input_size))
        self.W_ho = nn.Parameter(torch.empty(hidden_size, hidden_size))
        self.b_io = nn.Parameter(torch.zeros(hidden_size))
        self.b_ho = nn.Parameter(torch.zeros(hidden_size))

        self._reset_parameters()

    def _reset_parameters(self):
        # Kaiming uniform – same default initialisation as nn.Linear
        for w in [self.W_ii, self.W_hi, self.W_if, self.W_hf,
                  self.W_ig, self.W_hg, self.W_io, self.W_ho]:
            nn.init.kaiming_uniform_(w, a=math.sqrt(5))

    def forward(self, x_t, h_prev, c_prev):
        # matmul: x_t @ W.T → [B,I] @ [I,H] = [B,H]

        # — FORGET GATE —————
        # How much of the old cell state C_{t-1} should we keep?
        # sigmoid output (0,1): 0 = erase completely, 1 = keep completely
        f_t = torch.sigmoid(
            torch.matmul(x_t, self.W_if.T) + self.b_if +
            torch.matmul(h_prev, self.W_hf.T) + self.b_hf
        ) # [B, H]

        # — INPUT GATE —————
        # How much of the new candidate information should we write?
        i_t = torch.sigmoid(
            torch.matmul(x_t, self.W_ii.T) + self.b_ii +
            torch.matmul(h_prev, self.W_hi.T) + self.b_hi
        ) # [B, H]

        # — CANDIDATE CELL STATE —————
        # What new content do we propose writing to memory?
        # tanh output (-1,1): can add or subtract information

```

```

g_t = torch.tanh(
    torch.matmul(x_t, self.W_ig.T) + self.b_ig +
    torch.matmul(h_prev, self.W_hg.T) + self.b_hg
) # [B, H]

# — OUTPUT GATE —————
# Which memory units should be visible in the hidden state?
o_t = torch.sigmoid(
    torch.matmul(x_t, self.W_io.T) + self.b_io +
    torch.matmul(h_prev, self.W_ho.T) + self.b_ho
) # [B, H]

# — CELL STATE UPDATE —————
# Erase old memory + write new content
# Addition (not tanh!) lets gradients flow unchanged → no vanishing
c_t = f_t * c_prev + i_t * g_t # [B, H]

# — HIDDEN STATE —————
# Filtered view of the updated cell state
h_t = o_t * torch.tanh(c_t) # [B, H]

return h_t, c_t

def init_hidden(self, batch_size, device):
    h0 = torch.zeros(batch_size, self.hidden_size, device=device)
    c0 = torch.zeros(batch_size, self.hidden_size, device=device)
    return h0, c0

def extra_repr(self):
    return f'input_size={self.input_size}, hidden_size={self.hidden_size}'

# — Quick shape verification —————
_B, _I, _H = 4, 8, 16
_cell = CustomLSTMCell(input_size=_I, hidden_size=_H)
_x = torch.randn(_B, _I)
_h0, _c0 = _cell.init_hidden(_B, torch.device('cpu'))
_h1, _c1 = _cell(_x, _h0, _c0)

print('CustomLSTMCell – shape verification')
print(f' x_t : {tuple(_x.shape)} → input')
print(f' h_t : {tuple(_h1.shape)} → new hidden state')
print(f' c_t : {tuple(_c1.shape)} → new cell state')
print(f' h_t range: [{_h1.min():.3f}, {_h1.max():.3f}] (tanh-bounded)')
print('PASS')

CustomLSTMCell – shape verification
x_t : (4, 8) → input
h_t : (4, 16) → new hidden state
c_t : (4, 16) → new cell state
h_t range: [-0.300, 0.200] (tanh-bounded)
PASS

```

✓ Comparison with PyTorch `nn.LSTMCell`

We verify our custom cell is **numerically identical** to PyTorch's built-in by copying weights from `nn.LSTMCell` into `CustomLSTMCell` and comparing outputs.

How PyTorch stores weights (fused format)

PyTorch packs all 4 gate weights into **2 large matrices** for GPU efficiency:

```

weight_ih [4H, I] → rows 0:H = W_ii (input gate)
                  rows H:2H = W_if (forget gate)
                  rows 2H:3H = W_ig (candidate)
                  rows 3H:4H = W_io (output gate)

weight_hh [4H, H] → same layout for hidden-side weights
bias_ih   [4H]    → same layout for input biases
bias_hh   [4H]    → same layout for hidden biases

```

Gate order is always $i \rightarrow f \rightarrow g \rightarrow o$.

Steps

1. Create `nn.LSTMCell`, run it on random input
2. Slice `weight_ih`, `weight_hh`, `bias_ih`, `bias_hh` into per-gate tensors

3. Copy those slices into `CustomLSTMCell`
4. Run `CustomLSTMCell` on the same input
5. Compare with `torch.allclose` — differences should be $< 1e-5$ (float32 rounding only)

```

torch.manual_seed(0)
CMP_INPUT = 8
CMP_HIDDEN = 16
CMP_BATCH = 4
H = CMP_HIDDEN

# — Step 1: PyTorch reference cell —————
pytorch_cell = nn.LSTMCell(input_size=CMP_INPUT, hidden_size=CMP_HIDDEN)

# — Step 2: Identical random inputs —————
x_cmp = torch.randn(CMP_BATCH, CMP_INPUT)
h_cmp = torch.randn(CMP_BATCH, CMP_HIDDEN)
c_cmp = torch.randn(CMP_BATCH, CMP_HIDDEN)

# — Step 3: Run nn.LSTMCell —————
h_pytorch, c_pytorch = pytorch_cell(x_cmp, (h_cmp, c_cmp))

# — Step 4: Slice fused matrices —————
def slice_gate(matrix, gate_idx, H):
    '''Rows [gate_idx*H : (gate_idx+1)*H] belong to one gate.'''
    return matrix[gate_idx * H : (gate_idx + 1) * H].clone().detach()

# input-side weights [H, I]
W_ii = slice_gate(pytorch_cell.weight_ih, 0, H)
W_if = slice_gate(pytorch_cell.weight_ih, 1, H)
W_ig = slice_gate(pytorch_cell.weight_ih, 2, H)
W_io = slice_gate(pytorch_cell.weight_ih, 3, H)

# hidden-side weights [H, H]
W_hi = slice_gate(pytorch_cell.weight_hh, 0, H)
W_hf = slice_gate(pytorch_cell.weight_hh, 1, H)
W_hg = slice_gate(pytorch_cell.weight_hh, 2, H)
W_ho = slice_gate(pytorch_cell.weight_hh, 3, H)

# input-side biases [H]
b_ii = slice_gate(pytorch_cell.bias_ih, 0, H)
b_if = slice_gate(pytorch_cell.bias_ih, 1, H)
b_ig = slice_gate(pytorch_cell.bias_ih, 2, H)
b_io = slice_gate(pytorch_cell.bias_ih, 3, H)

# hidden-side biases [H]
b_hi = slice_gate(pytorch_cell.bias_hh, 0, H)
b_hf = slice_gate(pytorch_cell.bias_hh, 1, H)
b_hg = slice_gate(pytorch_cell.bias_hh, 2, H)
b_ho = slice_gate(pytorch_cell.bias_hh, 3, H)

# — Step 5: Copy into CustomLSTMCell —————
custom_cmp = CustomLSTMCell(input_size=CMP_INPUT, hidden_size=CMP_HIDDEN)

with torch.no_grad(): # required when writing directly to nn.Parameter tensors
    custom_cmp.W_ii.copy_(W_ii); custom_cmp.W_hi.copy_(W_hi)
    custom_cmp.W_if.copy_(W_if); custom_cmp.W_hf.copy_(W_hf)
    custom_cmp.W_ig.copy_(W_ig); custom_cmp.W_hg.copy_(W_hg)
    custom_cmp.W_io.copy_(W_io); custom_cmp.W_ho.copy_(W_ho)
    custom_cmp.b_ii.copy_(b_ii); custom_cmp.b_hi.copy_(b_hi)
    custom_cmp.b_if.copy_(b_if); custom_cmp.b_hf.copy_(b_hf)
    custom_cmp.b_ig.copy_(b_ig); custom_cmp.b_hg.copy_(b_hg)
    custom_cmp.b_io.copy_(b_io); custom_cmp.b_ho.copy_(b_ho)

# — Step 6: Run CustomLSTMCell —————
h_custom, c_custom = custom_cmp(x_cmp, h_cmp, c_cmp)

# — Step 7: Numerical comparison —————
h_diff = (h_pytorch.detach() - h_custom.detach()).abs()
c_diff = (c_pytorch.detach() - c_custom.detach()).abs()
h_ok = torch.allclose(h_pytorch.detach(), h_custom.detach(), atol=1e-5)
c_ok = torch.allclose(c_pytorch.detach(), c_custom.detach(), atol=1e-5)

print('=' * 55)
print(' nn.LSTMCell vs CustomLSTMCell')
print('=' * 55)
print(f' Hidden state max |diff| : {h_diff.max().item():.2e}')
print(f' Hidden state mean |diff| : {h_diff.mean().item():.2e}')
print(f' torch.allclose (h_t) : {h_ok}')
print(f' Cell state max |diff| : {c_diff.max().item():.2e}')
print(f' Cell state mean |diff| : {c_diff.mean().item():.2e}')

```

```

print(f' torch.allclose (c_t)      : {c_ok}')
print()
status = 'PASS' if (h_ok and c_ok) else 'FAIL'
print(f' RESULT: {status} – differences are float32 rounding only (< 1e-5)')

=====
nn.LSTMCell vs CustomLSTMCell
=====
Hidden state max |diff| : 2.98e-08
Hidden state mean |diff| : 4.69e-09
torch.allclose (h_t)    : True
Cell state max |diff| : 1.19e-07
Cell state mean |diff| : 1.07e-08
torch.allclose (c_t)    : True

RESULT: PASS – differences are float32 rounding only (< 1e-5)

```

Part B — Data Preprocessing

Sentiment140 Dataset

Property	Value
Total tweets	1,600,000
Negative (label 0)	800,000
Positive (label 4)	800,000
Language	English
Source	Twitter API (2009)

Important: The dataset is **sorted** — all 800k negatives first, then all 800k positives. Loading only the first N rows gives only negative samples. Fix: load N/2 rows from each half, then concatenate and shuffle.

Preprocessing steps

1. Load CSV with pandas
2. Clean tweets with regex
3. Tokenize (split on whitespace)
4. Build vocabulary (word frequency counting)
5. Encode words → integer IDs
6. Pad / truncate to fixed length
7. Split 60 / 20 / 20
8. Create PyTorch `Dataset` + `DataLoader`

```

# =====
# Download Sentiment140 from Kaggle
#
# Setup steps:
# 1. Go to kaggle.com > Account > API > Create New Token
# 2. This downloads kaggle.json to your computer
# 3. Run this cell and upload kaggle.json when prompted
# =====

import os
from google.colab import files

print('Upload your kaggle.json API key file:')
uploaded = files.upload() # opens a file-picker dialog

# Place the key where Kaggle CLI expects it
os.makedirs('/root/.kaggle', exist_ok=True)
with open('/root/.kaggle/kaggle.json', 'wb') as f:
    f.write(list(uploaded.values())[0])
os.chmod('/root/.kaggle/kaggle.json', 0o600)
print('Kaggle API configured.')

# Download and extract
print('Downloading Sentiment140 (238 MB)...')
!kaggle datasets download -d kazanova/sentiment140
!unzip -q sentiment140.zip
print('Done!')
!ls -lh training.1600000.processed.noemoticon.csv

```

Upload your kaggle.json API key file:

Choose Files training.160...moticon.csv

training.1600000.processed.noemoticon.csv(text/csv) - 238803811 bytes, last modified: 5/23/2026 - 100% done

Saving training.1600000.processed.noemoticon.csv to training.1600000.processed.noemoticon.csv

Kaggle API configured.

Downloading Sentiment140 (238 MB)...

```
Traceback (most recent call last):
```

File "/usr/local/bin/kaggle", line 4, in <module>

```
from kaggle.cli import main
```

File `"/usr/local/lib/python3.12/dist-packages/kaggle/__init__.py"`, line 10, in `<module>`

```
api.authenticate()
```

File "/usr/local/lib/python3.12/dist-packages/kaggle/api/kaggle_api_extended.py", line 763, in authenticate

```
if self._authenticate_with_legacy_apikey():
```

AA

```
File "/usr/local/lib/python3.12/dist-packages/kaggle/api/kaggle_api_extended.py", line 808, in _authenticate_with_legacy_credentials
    raise ValueError("Error: Missing %s in configuration." % item)
```

```
ValueError: Error: Missing username in configuration.
```

```
unzip: cannot find or open sentiment140.zip, sentiment140.zip.zip or sentiment140.zip.ZIP.
```

Done!

```
-rw-r--r-- 1 root root 228M May 23 11:19 training.1600000.processed.noemoticon.csv
```

```
# =====
# Load Sentiment140 CSV
# Columns: polarity, id, date, flag, user, text (no header row in file)
# =====
```

```
CSV_PATH = 'training.1600000.processed.noemoticon.csv'
N_ROWS   = 100_000 # 50k negative + 50k positive; set None for all 1.6M
TOTAL ROWS = 1 600 000 # full dataset size
```

```
col_names = ['polarity', 'id', 'date', 'flag', 'user', 'text']
```

```
# Load equal halves so labels are balanced even at small N_ROWS
half = N_ROWS // 2
```

[illegible]

```
pos_df = pd.read_csv(CSV_PATH, encoding='latin-1', header=None,
                     names=col_names, skiprows=TOTAL_ROWS // 2, nrows=half)
```

```
df = pd.concat([neg_df, pos_df], ignore_index=True)
df = df.sample(frac=1, random_state=42).reset_index(drop=True) # shuffle
```

```
# Keep only polarity and text; remap 4 -> 1
df = df[['polarity', 'text']].copy()
df['label'] = df['polarity'].map({0: 0, 4: 1})
df.drop(columns=['polarity'], inplace=True)
```

```
print(f'Rows loaded : {len(df):,}')
print(f'\nLabel counts:')
print(df['label'].value_counts().to_string())
print(f'\nSample rows:')
print(df[['text', 'label']].head(5).to_string(index=False))
```

Rows loaded : 100,000

Label counts:

label

1 50000

0 50000

Sample rows:

home from L&L, would rather work at night, much less of
@mr_billiam ooh...thanks for the tip. I'll do that ASAP. You are
@belle_lulu site was hacked but thank you, I shall think of that area. I'm actually going to do it through Tesco Deals, che
back to repeating the climb by hannah montana, i'm done repeating the world's greatest by r.
@stunninwmylove 562 262 1736 call meeee mikey wiz

- ✓ Data Cleaning

Raw tweets contain noise that hurts model learning:

Noise type	Raw example	After cleaning
@mention	@elonmusk great!	great!
URL	check http://t.co/abc	check
Hashtag #	feeling #happy	feeling happy
Punctuation	can't!!! wow...	cant wow

Noise type	Raw example	After cleaning
Mixed case	GREAT day	great day
Extra spaces	too many spaces	too many spaces

Order matters

1. Remove URLs **first** — they contain `#` and `.` that would confuse later steps
2. Remove `@mentions` — they sometimes precede text we want to keep
3. Strip `#` symbol but **keep the word** — hashtag text carries sentiment
4. Lowercase **after** URL removal — URLs are case-sensitive
5. Remove punctuation **last** — after already handling `#` and `@`

```
# =====
# Tweet Cleaning - pre-compiled regex patterns (compiled c
# =====

_URL_PAT = re.compile(r'https?://\S+|www.\S+') # ht
_MENTION_PAT = re.compile(r'@\w+') # @
_HASHTAG_PAT = re.compile(r'#(\w+)') # #
_PUNCT_PAT = re.compile(r'^\w\s') # a
_SPACE_PAT = re.compile(r'\s+') # #

def clean_tweet(text):
    text = _URL_PAT.sub('', text) # step 1: remc
    text = _MENTION_PAT.sub('', text) # step 2: remc
    text = _HASHTAG_PAT.sub(r'\1', text) # step 3: #wor
    text = text.lower() # step 4: lowe
    text = _PUNCT_PAT.sub(' ', text) # step 5: remc
    text = _SPACE_PAT.sub(' ', text) # step 6: coll
    return text.strip() # step 7: trim

# Apply to every tweet
print('Cleaning tweets...')
df['clean_text'] = df['text'].astype(str).apply(clean_twee

# Drop tweets that are now empty (were URLs / mentions onl
before = len(df)
df = df[df['clean_text'].str.len() > 0].reset_index(drop=T
print(f'Dropped {before - len(df)} empty tweets. Remaining

# Before / after examples
print('\nBefore vs After Cleaning:')
print('-' * 70)
for i in range(3):
    print(f'RAW : {df["text"].iloc[i]}')
    print(f'CLEAN : {df["clean_text"].iloc[i]}')
    print()
```

"@username" is not an allowed annotation - allowed values include
[`@param`, `@title`, `@markdown`].

[Hide code](#)

```
Cleaning tweets...
Dropped 197 empty tweets. Remaining: 99,803
```

Before vs After Cleaning:

```
-----
RAW : home from L&L, would rather work at night, much less of a rush
CLEAN : home from l amp l would rather work at night much less of a rush
```

```
RAW : @mr_billiam ooh...thanks for the tip. I'll do that ASAP. You are a star
CLEAN : ooh thanks for the tip i ll do that asap you are a star
```

```
RAW : @belle_lulu site was hacked but thank you, I shall think of that area. I'm actually going to do it through Tesco De
CLEAN : site was hacked but thank you i shall think of that area i m actually going to do it through tesco deals cheaper x
```

Tokenization & Vocabulary Building

Tokenization

Split each cleaned tweet on whitespace → list of word tokens.

```
"love this movie" → ["love", "this", "movie"]
```

Vocabulary (word → integer ID)


```

<PAD> → 0    # padding token – always index 0
<UNK> → 1    # unknown words – always index 1
the   → 2    # most frequent word gets index 2
i     → 3
...
love  → 52

```

Why `min_freq = 2`?

Words appearing only once are usually:

- Typos (`teh`, `hte`)
- Rare proper nouns (`@JohnSmith42`) They don't help generalisation. Excluding them shrinks the vocabulary and speeds up training.

Encoding + Padding

```
["love", "this", "xyz"] → [52, 7, 1]    # xyz is UNK (id=1)
```

Pad to `MAX_LEN=30`:

```
[52, 7, 1, 0, 0, 0, ..., 0]           # 27 zeros appended
```

Critical rule: vocabulary built on TRAINING DATA ONLY

If we build the vocab on all data and then split, val/test word frequencies leak into the vocabulary — this is **data leakage** that inflates accuracy.

```

# =====
# Vocabulary
# =====

PAD_TOKEN = '<PAD>'    # always index 0
UNK_TOKEN = '<UNK>'    # always index 1

class Vocabulary:
    """
    Maps words <-> integer IDs.

    Special tokens are fixed:
    <PAD> -> 0    (used to fill short sequences)
    <UNK> -> 1    (used for words not seen in training)
    """

    def __init__(self, min_freq=2):
        self.min_freq = min_freq
        self.word2idx = {PAD_TOKEN: 0, UNK_TOKEN: 1}
        self.idx2word = {0: PAD_TOKEN, 1: UNK_TOKEN}

    def build(self, tokenized_series):
        """
        Count all words across the training split and add
        those appearing >= min_freq times to the vocabulary.
        """
        counter = Counter()
        for tokens in tokenized_series:
            counter.update(tokens)

        # most_common() is sorted by frequency descending
        # so high-frequency words get low indices
        for word, freq in counter.most_common():
            if freq < self.min_freq:
                break                # all remaining have freq < min_freq
            if word not in self.word2idx:
                idx = len(self.word2idx)
                self.word2idx[word] = idx
                self.idx2word[idx] = word

        print('Vocabulary built:')
        print(f' Unique words seen      : {len(counter):,}')
        print(f' Words kept (>={self.min_freq}) : {len(self.word2idx) - 2:,}')
        print(f' Vocabulary size          : {len(self.word2idx):,} (incl. PAD + UNK)')

    def encode(self, tokens):
        """List of word strings -> list of integer IDs."""
        unk = self.word2idx[UNK_TOKEN]

```

```

        return [self.word2idx.get(w, unk) for w in tokens]

    def decode(self, ids):
        '''List of integer IDs -> list of word strings.'''
        return [self.idx2word.get(i, UNK_TOKEN) for i in ids]

    def __len__(self):
        return len(self.word2idx)

# =====
# Tokenize + Encode + Pad
# =====

MAX_LEN = 30 # sequence length (median tweet = 12 tokens, nearly all fit in 30)
MIN_FREQ = 2 # vocabulary frequency threshold

# Tokenize: split cleaned text on whitespace
df['tokens'] = df['clean_text'].apply(str.split)

lengths = df['tokens'].apply(len)
print('Token length statistics:')
print(f' Min      : {lengths.min()}')
print(f' Max      : {lengths.max()}')
print(f' Mean      : {lengths.mean():.1f}')
print(f' Median    : {int(lengths.median())}')
pct = (lengths <= MAX_LEN).mean() * 100
print(f' Tweets fitting in {MAX_LEN} tokens : {pct:.1f}%')
print()

# Vocabulary object (will be fitted on training split only – see next cell)
vocab = Vocabulary(min_freq=MIN_FREQ)

def encode_and_pad(token_series, vocab, max_len):
    '''
    Encode token lists to integer IDs, truncate/pad to max_len.

    Returns:
        LongTensor [N, max_len]
    '''
    pad_id = vocab.word2idx[PAD_TOKEN] # = 0
    rows = []
    for tokens in token_series:
        ids = vocab.encode(tokens) # variable-length list of ints
        ids = ids[:max_len] # truncate if > max_len
        ids += [pad_id] * (max_len - len(ids)) # pad with 0s if < max_len
        rows.append(ids)
    return torch.tensor(rows, dtype=torch.long) # [N, max_len]

print('Tokenization done. Vocabulary will be fitted after splitting.')

```

```

Token length statistics:
Min      : 1
Max      : 56
Mean      : 13.1
Median    : 12
Tweets fitting in 30 tokens : 99.6%

```

```
Tokenization done. Vocabulary will be fitted after splitting.
```

✓ Data Splitting — 60 / 20 / 20

Two-step split with `train_test_split`

```

Full data [100%]
|
├─ test      [20%]      Step 1: test_size = 0.20
|
└─ trainval  [80%]
    |
    ├─ train  [60%]      Step 2: test_size = 0.20 / 0.80 = 0.25
    └─ val    [20%]

```

Why `stratify=label`?

Without stratification, random chance could give one split 48% positives and another 52% — a small but real difference that biases reported metrics. `stratify=label` guarantees **exactly 50% positive / 50% negative** in every split.

Why split before building the vocabulary?

Building the vocab after the split, on **training tokens only**, prevents val/test word information from influencing which words enter the vocabulary. This is a subtle but important form of **data leakage** to avoid.

```
# =====
# Step 1: 60 / 20 / 20 split
# =====

# Carve out 20% test set
trainval_df, test_df = train_test_split(
    df,
    test_size=0.20,
    random_state=42,
    stratify=df['label'],          # preserve 50/50 balance
)

# Split remaining 80% into 60% train + 20% val
# 0.20 / 0.80 = 0.25 of the trainval pool = 20% of the full dataset
train_df, val_df = train_test_split(
    trainval_df,
    test_size=0.25,
    random_state=42,
    stratify=trainval_df['label'],
)

train_df = train_df.reset_index(drop=True)
val_df   = val_df.reset_index(drop=True)
test_df  = test_df.reset_index(drop=True)

total = len(df)
print('Split sizes:')
print(f' Total : {total:}, (100.0%)')
print(f' Train  : {len(train_df):}, ({100*len(train_df)/total:.1f}%)')
print(f' Val    : {len(val_df):}, ({100*len(val_df)/total:.1f}%)')
print(f' Test   : {len(test_df):}, ({100*len(test_df)/total:.1f}%)')

print('\nClass balance (% positive):')
for name, split in [('Train', train_df), ('Val ', val_df), ('Test ', test_df)]:
    pct = split['label'].mean() * 100
    print(f' {name} : {pct:.1f}% positive')

# =====
# Step 2: Build vocabulary – TRAINING DATA ONLY
# =====
print()
print('Building vocabulary on training data only...')
vocab.build(train_df['tokens'])

# =====
# Step 3: Encode + pad all three splits using the SAME vocabulary
# =====
print()
print('Encoding and padding...')
train_ids = encode_and_pad(train_df['tokens'], vocab, MAX_LEN)
val_ids   = encode_and_pad(val_df['tokens'],  vocab, MAX_LEN)
test_ids  = encode_and_pad(test_df['tokens'],  vocab, MAX_LEN)

train_labels = torch.tensor(train_df['label'].values, dtype=torch.float32)
val_labels   = torch.tensor(val_df['label'].values,  dtype=torch.float32)
test_labels  = torch.tensor(test_df['label'].values, dtype=torch.float32)

print(f' train_ids : {tuple(train_ids.shape)}')
print(f' val_ids   : {tuple(val_ids.shape)}')
print(f' test_ids  : {tuple(test_ids.shape)}')

# Round-trip decode check
sample_tok = train_df['tokens'].iloc[0]
sample_ids = train_ids[0].tolist()
decoded    = vocab.decode([i for i in sample_ids if i != 0])
print(f'\nRound-trip check (train[0]):')
print(f' Tokens : {sample_tok}')
print(f' IDs    : {sample_ids}')
print(f' Decoded : {decoded}')
```

Split sizes:

```
Total : 99,803 (100.0%)
Train  : 59,881 (60.0%)
```

```
Val      : 19,961 (20.0%)  
Test     : 19,961 (20.0%)  
  
Class balance (% positive):  
Train    : 50.0% positive  
Val      : 50.0% positive  
Test     : 50.0% positive  
  
Building vocabulary on training data only...  
Vocabulary built:  
Unique words seen      : 38,950  
Words kept (>=2)      : 16,553  
Vocabulary size        : 16,555 (incl. PAD + UNK)  
  
Encoding and padding...  
train_ids : (59881, 30)  
val_ids   : (19961, 30)  
test_ids  : (19961, 30)  
  
Round-trip check (train[0]):  
Tokens : ['loves', 'bjork', 's', 'unravel', 'btw', 'radiohead', 'made', 'a', 'cover', 'to', 'that', 'song', 'gt', 'both',  
IDs      : [611, 7761, 14, 1, 504, 9223, 213, 5, 1719, 3, 17, 327, 427, 513, 332, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
Decoded : ['loves', 'bjork', 's', '<UNK>', 'btw', 'radiohead', 'made', 'a', 'cover', 'to', 'that', 'song', 'gt', 'both', '
```

- PyTorch Dataset & DataLoader

TweetDataset

A `Dataset` wraps our tensors. PyTorch's `DataLoader` calls `__getitem__` repeatedly to collect samples into a batch.

DataLoader settings

Loader	shuffle	Reason
train_loader	True	Prevents model learning sample order
val_loader	False	Consistent results for comparison
test_loader	False	Reproducible final evaluation

Batch tensor shapes

```
tweet_batch : [batch_size, MAX_LEN]    - LongTensor of token IDs
label_batch : [batch_size]              - FloatTensor of 0.0 / 1.0
```

```
# =====
# TweetDataset
# =====

class TweetDataset(Dataset):
    """
    Wraps (token_ids, labels) tensors for use with DataLoader.

    __getitem__ returns one (tweet, label) pair.
    DataLoader calls this repeatedly and stacks them into a batch.
    """

    def __init__(self, token_ids, labels):
        assert len(token_ids) == len(labels), 'size mismatch'
        self.token_ids = token_ids # [N, MAX_LEN] LongTensor
        self.labels = labels # [N] FloatTensor

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.token_ids[idx], self.labels[idx]
        # returns: ([MAX_LEN] LongTensor, scalar FloatTensor)

# =====
# DataLoaders
# =====

BATCH_SIZE = 128
NUM_WORKERS = 2 # parallel loading; set 0 if you see errors

train_dataset = TweetDataset(train_ids, train_labels)
val_dataset = TweetDataset(val_ids, val_labels)
test_dataset = TweetDataset(test_ids, test_labels)
```

```

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
                           shuffle=True, num_workers=NUM_WORKERS,
                           pin_memory=(device.type == 'cuda'))

val_loader   = DataLoader(val_dataset, batch_size=BATCH_SIZE,
                           shuffle=False, num_workers=NUM_WORKERS)

test_loader  = DataLoader(test_dataset, batch_size=BATCH_SIZE,
                           shuffle=False, num_workers=NUM_WORKERS)

# Verify batch shapes
sample_tw, sample_lb = next(iter(train_loader))
print('DataLoaders ready:')
print(f' train : {len(train_loader)} batches ({len(train_dataset):,} samples)')
print(f' val  : {len(val_loader)} batches ({len(val_dataset):,} samples)')
print(f' test : {len(test_loader)} batches ({len(test_dataset):,} samples)')
print()
print('One batch shapes:')
print(f' tweet_batch : {tuple(sample_tw.shape)} dtype={sample_tw.dtype}')
print(f' label_batch : {tuple(sample_lb.shape)} dtype={sample_lb.dtype}')

```

```

DataLoaders ready:
train : 468 batches (59,881 samples)
val   : 156 batches (19,961 samples)
test  : 156 batches (19,961 samples)

One batch shapes:
tweet_batch : (128, 30) dtype=torch.int64
label_batch : (128,) dtype=torch.float32

```

Part C — Sentiment Analysis Model

Architecture

```

token_ids [batch, seq_len]
|
▼ nn.Embedding (vocab_size → embed_dim)
embeds [batch, seq_len, embed_dim]
|
▼ Dropout
|
▼ CustomLSTMCell × seq_len (unrolled)
|   at each step t:
|       x_t = embeds[:, t, :]           [batch, embed_dim]
|       h_t, c_t = cell(x_t, h, c)      [batch, hidden_size]
|
▼ h_T (last hidden state) [batch, hidden_size]
|
▼ Dropout
|
▼ nn.Linear (hidden_size → 1)
|
logit [batch, 1] — BCEWithLogitsLoss applies sigmoid internally

```

Why the last hidden state?

The LSTM processes the tweet word by word. At the final step T , **h_T has seen every word** and summarises the whole sentence. Using h_T for classification is the standard approach for sequence classification.

Why `BCEWithLogitsLoss`?

It fuses `sigmoid + binary cross-entropy` in one numerically stable operation. Calling `sigmoid` in the model and `BCELoss` separately can cause NaN for very large / small logits. `BCEWithLogitsLoss` avoids this.

```

# =====
# SentimentLSTM
# =====

class SentimentLSTM(nn.Module):
    ...
    Binary sentiment classifier:
        Embedding --> CustomLSTMCell (unrolled) --> Linear --> logit

    Args:

```

```

vocab_size : size of vocabulary (for nn.Embedding)
embed_dim  : word embedding dimension
hidden_size : LSTM hidden state dimension
n_layers   : number of stacked LSTM layers
dropout    : dropout rate applied after embedding and between layers
...

def __init__(self, vocab_size, embed_dim, hidden_size,
              n_layers=1, dropout=0.3):
    super().__init__()
    self.hidden_size = hidden_size
    self.n_layers    = n_layers

    # Word embeddings: token ID -> dense vector
    # padding_idx=0 keeps the PAD embedding as all-zeros and never updates it
    self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
    self.drop      = nn.Dropout(dropout)

    # Stack of CustomLSTMCells
    # Layer 0 receives embed_dim input; deeper layers receive hidden_size
    self.lstm_cells = nn.ModuleList([
        CustomLSTMCell(
            input_size = embed_dim if layer == 0 else hidden_size,
            hidden_size = hidden_size
        )
        for layer in range(n_layers)
    ])

    # Final linear classifier
    self.fc = nn.Linear(hidden_size, 1)

def forward(self, token_ids):
    """
    Args:
        token_ids : LongTensor [B, T]  B=batch, T=seq_len

    Returns:
        logits : FloatTensor [B, 1]  (raw score; apply sigmoid for prob)
    """
    B, T = token_ids.shape
    dev = token_ids.device

    # Embed token IDs -> [B, T, embed_dim]
    x = self.drop(self.embedding(token_ids))

    # Initialise hidden and cell states to zero for each layer
    h = [torch.zeros(B, self.hidden_size, device=dev) for _ in range(self.n_layers)]
    c = [torch.zeros(B, self.hidden_size, device=dev) for _ in range(self.n_layers)]

    # Unroll LSTM over the sequence one step at a time
    for t in range(T):
        inp = x[:, t, :] # [B, embed_dim] - current token
        for l, cell in enumerate(self.lstm_cells):
            h[l], c[l] = cell(inp, h[l], c[l])
            inp = self.drop(h[l]) # feed h as input to the next layer

    # Classify using the last layer's final hidden state
    logits = self.fc(self.drop(h[-1])) # [B, 1]
    return logits

# — Build model —
VOCAB_SIZE = len(vocab)
EMBED_DIM  = 64
HIDDEN_SIZE = 128
N_LAYERS   = 1
DROPOUT    = 0.3

model = SentimentLSTM(
    vocab_size = VOCAB_SIZE,
    embed_dim  = EMBED_DIM,
    hidden_size = HIDDEN_SIZE,
    n_layers   = N_LAYERS,
    dropout    = DROPOUT,
).to(device)

total_p = sum(p.numel() for p in model.parameters())
trainable_p = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(model)
print(f'\nTotal parameters      : {total_p:,}')
print(f'Trainable parameters : {trainable_p:,}')

```

```
# Forward-pass sanity check
_dummy = torch.randint(0, VOCAB_SIZE, (4, MAX_LEN)).to(device)
_out = model(_dummy)
print(f'\nForward pass: {tuple(_dummy.shape)} -> {tuple(_out.shape)} OK')
```

```
SentimentLSTM(
  (embedding): Embedding(16555, 64, padding_idx=0)
  (drop): Dropout(p=0.3, inplace=False)
  (lstm_cells): ModuleList(
    (0): CustomLSTMCell(input_size=64, hidden_size=128)
  )
  (fc): Linear(in_features=128, out_features=1, bias=True)
)
```

Total parameters : 1,158,977

Trainable parameters : 1,158,977

Forward pass: (4, 30) -> (4, 1) OK

```
# =====
# Loss, Optimizer, Scheduler
# =====

EPOCHS = 10 # number of full passes through training data
LR = 1e-3 # Adam learning rate
MAX_NORM = 1.0 # gradient-clipping max norm

# BCEWithLogitsLoss: sigmoid + binary cross-entropy, numerically stable
criterion = nn.BCEWithLogitsLoss()

# Adam: per-parameter adaptive learning rates, well-suited for NLP
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

# Reduce LR by half if val_loss doesn't improve for 2 epochs
# Note: verbose= was removed in PyTorch 2.2 – use get_last_lr() to print manually
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', patience=2, factor=0.5
)

print('Training configuration')
print(f' Epochs : {EPOCHS}')
print(f' Batch size : {BATCH_SIZE}')
print(f' Learning rate : {LR}')
print(f' Optimizer : Adam')
print(f' Loss : BCEWithLogitsLoss')
print(f' Scheduler : ReduceLROnPlateau (patience=2)')
print(f' Grad clipping : max_norm={MAX_NORM}')
print()
print('Model hyperparameters')
print(f' Vocab size : {VOCAB_SIZE:,}')
print(f' Embed dim : {EMBED_DIM}')
print(f' Hidden size : {HIDDEN_SIZE}')
print(f' LSTM layers : {N_LAYERS}')
print(f' Dropout : {DROPOUT}')
print(f' Max seq length : {MAX_LEN}')
print(f' Device : {device}')
```

```
Training configuration
Epochs : 10
Batch size : 128
Learning rate : 0.001
Optimizer : Adam
Loss : BCEWithLogitsLoss
Scheduler : ReduceLROnPlateau (patience=2)
Grad clipping : max_norm=1.0
```

```
Model hyperparameters
Vocab size : 16,555
Embed dim : 64
Hidden size : 128
LSTM layers : 1
Dropout : 0.3
Max seq length : 30
Device : cuda
```

```
# =====
# Training and Evaluation Functions
# =====

def train_epoch(model, loader, criterion, optimizer, device, max_norm):
    """
    One full pass over the training DataLoader.

    Returns:
```

```

    avg_loss : float - mean loss per sample
    accuracy : float - fraction of correct predictions
'''
model.train() # activates Dropout
total_loss, total_correct, n = 0.0, 0, 0

for tweets, labels in loader:
    tweets = tweets.to(device) # [B, T]
    labels = labels.to(device).unsqueeze(1) # [B, 1]

    optimizer.zero_grad()
    logits = model(tweets) # [B, 1]
    loss = criterion(logits, labels)

    loss.backward()

    # Gradient clipping: if the global grad norm > max_norm,
    # scale all gradients down so the norm equals max_norm
    nn.utils.clip_grad_norm_(model.parameters(), max_norm)

    optimizer.step()

    preds = (torch.sigmoid(logits) >= 0.5).float()
    total_correct += (preds == labels).sum().item()
    n += labels.size(0)
    total_loss += loss.item() * labels.size(0)

return total_loss / n, total_correct / n

def evaluate(model, loader, criterion, device):
    '''
    Evaluate model on a DataLoader without updating weights.

    Returns:
        avg_loss : float
        accuracy : float
        all_preds : list of int (0 or 1 predictions)
        all_labels : list of int (0 or 1 true labels)
    '''
    model.eval() # disables Dropout
    total_loss, total_correct, n = 0.0, 0, 0
    all_preds, all_labels = [], []

    with torch.no_grad(): # no gradient computation
        for tweets, labels in loader:
            tweets = tweets.to(device)
            labels = labels.to(device).unsqueeze(1)

            logits = model(tweets)
            loss = criterion(logits, labels)

            preds = (torch.sigmoid(logits) >= 0.5).float()
            total_correct += (preds == labels).sum().item()
            n += labels.size(0)
            total_loss += loss.item() * labels.size(0)

            all_preds.extend(preds.cpu().squeeze().tolist())
            all_labels.extend(labels.cpu().squeeze().tolist())

    return total_loss / n, total_correct / n, all_preds, all_labels

print('train_epoch() and evaluate() defined.')
print()
print('Training step summary:')
print(' 1. model.train()          enable Dropout')
print(' 2. forward pass            compute logits [B,1]')
print(' 3. BCEWithLogitsLoss        compute scalar loss')
print(' 4. loss.backward()          compute gradients')
print(' 5. clip_grad_norm_()        prevent gradient explosion')
print(' 6. optimizer.step()         update all parameters')

```

train_epoch() and evaluate() defined.

Training step summary:

1. model.train() enable Dropout
2. forward pass compute logits [B,1]
3. BCEWithLogitsLoss compute scalar loss
4. loss.backward() compute gradients
5. clip_grad_norm_() prevent gradient explosion
6. optimizer.step() update all parameters


```
# =====
# Full Training Loop
# =====

train_losses, val_losses = [], []
train_accs, val_accs = [], []
best_val_acc = 0.0

print('Starting training...')
print('=' * 72)
print(f'  {"Epoch":>5}  {"Train Loss":>10}  {"Train Acc":>9}  '
      f'{"Val Loss":>8}  {"Val Acc":>8}  {"Time":>6}')
print('=' * 72)

for epoch in range(1, EPOCHS + 1):
    t0 = time.time()

    # — Training pass —————
    tr_loss, tr_acc = train_epoch(
        model, train_loader, criterion, optimizer, device, MAX_NORM
    )

    # — Validation pass —————
    va_loss, va_acc, _, _ = evaluate(model, val_loader, criterion, device)

    elapsed = time.time() - t0

    # Store metrics for plotting
    train_losses.append(tr_loss); val_losses.append(va_loss)
    train_accs.append(tr_acc); val_accs.append(va_acc)

    # Adjust learning rate based on validation loss
    scheduler.step(va_loss)

    # Save the best model weights
    tag = ''
    if va_acc > best_val_acc:
        best_val_acc = va_acc
        torch.save(model.state_dict(), 'best_model.pt')
        tag = ' <-- best'

    print(f'  {epoch:>5}  {tr_loss:>10.4f}  {tr_acc*100:>8.2f}%  '
          f'  {va_loss:>8.4f}  {va_acc*100:>7.2f}%  {elapsed:>5.1f}s{tag}')

print('=' * 72)
print(f'Training complete.  Best val accuracy: {best_val_acc*100:.2f}%')

# Restore the best weights for evaluation
model.load_state_dict(torch.load('best_model.pt', map_location=device))
print('Best model weights loaded.')
```

Starting training...

```
=====
Epoch  Train Loss  Train Acc  Val Loss  Val Acc  Time
=====
   1    0.6304      64.21%    0.5544    72.16%   33.3s <-- best
   2    0.5457      72.85%    0.5122    74.63%   32.6s <-- best
   3    0.5058      75.74%    0.4959    76.10%   31.9s <-- best
   4    0.4828      77.14%    0.4816    77.10%   33.1s <-- best
   5    0.4623      78.46%    0.4904    77.35%   31.7s <-- best
   6    0.4468      79.57%    0.4724    77.94%   33.1s <-- best
   7    0.4306      80.15%    0.4789    78.20%   32.1s <-- best
   8    0.4183      81.20%    0.4681    77.88%   32.1s
   9    0.4057      81.72%    0.4933    77.95%   32.7s
  10    0.3944      82.43%    0.4674    78.34%   31.7s <-- best
=====
```

Training complete. Best val accuracy: 78.34%
Best model weights loaded.

Part D — Training Visualization

Four curves to examine

Plot	What to look for
Train loss	Should decrease each epoch
Val loss	Should decrease too; rising = overfitting
Train accuracy	Should increase toward 100%
Val accuracy	True measure of generalisation

Diagnosing issues

- **Train loss ↓, Val loss ↑** → overfitting; increase dropout or reduce model size
- **Both losses plateau early** → underfitting; train longer or increase capacity
- **Val accuracy ≈ Train accuracy** → good generalisation

```
# =====
# Training Curves – Loss and Accuracy
# =====

epochs_x = list(range(1, EPOCHS + 1))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Training History', fontsize=15, fontweight='bold', y=1.01)

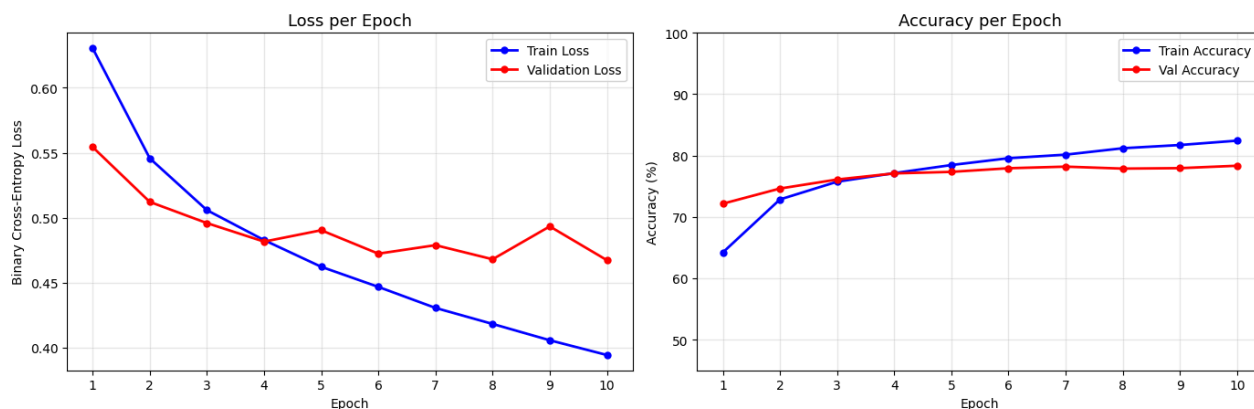
# — Loss —————
ax1.plot(epochs_x, train_losses, 'b-o', label='Train Loss', linewidth=2, markersize=5)
ax1.plot(epochs_x, val_losses, 'r-o', label='Validation Loss', linewidth=2, markersize=5)
ax1.set_title('Loss per Epoch', fontsize=13)
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Binary Cross-Entropy Loss')
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.set_xticks(epochs_x)

# — Accuracy —————
ax2.plot(epochs_x, [a*100 for a in train_accs], 'b-o',
        label='Train Accuracy', linewidth=2, markersize=5)
ax2.plot(epochs_x, [a*100 for a in val_accs], 'r-o',
        label='Val Accuracy', linewidth=2, markersize=5)
ax2.set_title('Accuracy per Epoch', fontsize=13)
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy (%)')
ax2.legend()
ax2.grid(True, alpha=0.3)
ax2.set_xticks(epochs_x)
ax2.set_ylim([45, 100])

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: training_curves.png')

# — Summary table —————
print(f'\n{"Epoch":>5} {"Train Loss":>10} {"Train Acc":>9} '
      f'{"Val Loss":>9} {"Val Acc":>8}')
print('-' * 48)
for i in range(EPOCHS):
    print(f'{i+1:>5} {train_losses[i]:>10.4f} {train_accs[i]*100:>8.2f}% '
          f'{val_losses[i]:>9.4f} {val_accs[i]*100:>7.2f}%')
```

Training History



Saved: training_curves.png

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.6304	64.21%	0.5544	72.16%
2	0.5457	72.85%	0.5122	74.63%
3	0.5058	75.74%	0.4959	76.10%
4	0.4828	77.14%	0.4816	77.10%
5	0.4623	78.46%	0.4904	77.35%
6	0.4468	79.57%	0.4724	77.94%
7	0.4306	80.15%	0.4789	78.20%
8	0.4183	81.20%	0.4681	77.88%
9	0.4057	81.72%	0.4933	77.95%
10	0.3944	82.43%	0.4674	78.34%

Part E — Final Evaluation on Test Set

The **test set** was held out during all training and model selection. It gives the most honest estimate of real-world performance.

Metrics

Metric	Formula	Meaning
Accuracy	$(TP+TN) / \text{total}$	Overall correctness
Precision	$TP / (TP+FP)$	When we predict positive, how often are we right?
Recall	$TP / (TP+FN)$	Of all actual positives, how many did we catch?
F1	$2 \cdot P \cdot R / (P+R)$	Harmonic mean of precision and recall

Confusion matrix

Actual	Predicted	
	Neg	Pos
	Neg [TN FP]	Pos [FN TP]

```
# =====
# Test Set Evaluation
# =====

print('Evaluating on test set...')
test_loss, test_acc, test_preds, test_true = evaluate(
    model, test_loader, criterion, device
)

print(f'Test Loss      : {test_loss:.4f}')
print(f'Test Accuracy : {test_acc*100:.2f}%')

# =====
# Confusion Matrix
# =====

y_true = [int(l) for l in test_true]
y_pred = [int(p) for p in test_preds]

cm = confusion_matrix(y_true, y_pred)
```

```

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=['Negative', 'Positive'],
    yticklabels=['Negative', 'Positive'],
    linewidths=0.5, ax=ax
)
ax.set_xlabel('Predicted Label', fontsize=12)
ax.set_ylabel('True Label',      fontsize=12)
ax.set_title('Confusion Matrix – Test Set', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: confusion_matrix.png')

tn, fp, fn, tp = cm.ravel()
print(f'\nConfusion matrix breakdown:')
print(f' True Negatives (TN) : {tn:,}')
print(f' False Positives (FP) : {fp:,}')
print(f' False Negatives (FN) : {fn:,}')
print(f' True Positives (TP) : {tp:,}')

# =====
# Classification Report
# =====

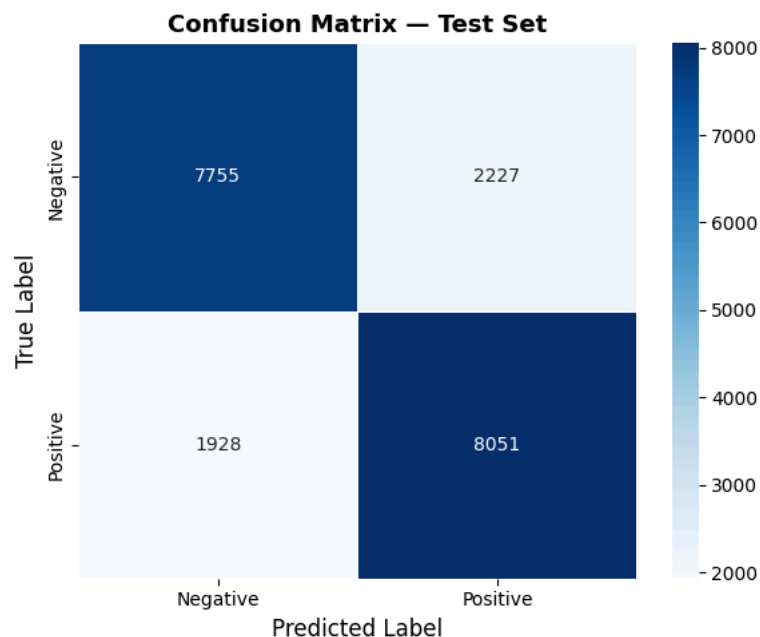
print('\nClassification Report:')
print(classification_report(
    y_true, y_pred,
    target_names=['Negative', 'Positive'],
    digits=4
))

```

Evaluating on test set...

Test Loss : 0.4516

Test Accuracy : 79.18%



Saved: confusion_matrix.png

Confusion matrix breakdown:

True Negatives (TN) : 7,755

False Positives (FP) : 2,227

False Negatives (FN) : 1,928

True Positives (TP) : 8,051

Classification Report:

	precision	recall	f1-score	support
Negative	0.8009	0.7769	0.7887	9982
Positive	0.7833	0.8068	0.7949	9979
accuracy			0.7918	19961
macro avg	0.7921	0.7918	0.7918	19961
weighted avg	0.7921	0.7918	0.7918	19961

Conclusion

What was built

Component	Details
CustomLSTMCell	8 weight matrices, 8 bias vectors; raw matmul + sigmoid + tanh
Equivalence proof	<code>torch.allclose</code> vs <code>nn.LSTMCell</code> with copied weights
Tokenizer	Regex cleaning + whitespace split — no external library
Vocabulary	Word-frequency counter with <code>min_freq=2</code>
Data pipeline	Balanced 60/20/20 stratified split; PAD/UNK encoding
SentimentLSTM	Embedding → CustomLSTMCell (unrolled) → Linear
Training	Adam + ReduceLROnPlateau + gradient clipping
Evaluation	Accuracy, confusion matrix, precision/recall/F1

Key insights

- 1. LSTM gates solve vanishing gradients** `C_t = f_t * C_prev + i_t * g_t` — the cell state is updated by addition, not matrix multiplication, so gradients flow unchanged over hundreds of timesteps.
- 2. Data leakage is subtle** Building the vocabulary on all data before splitting leaks val/test word frequencies into training. Always split first, then fit preprocessing on train only.
- 3. Stratification preserves class balance** `stratify=label` guarantees the same 50/50 class ratio in every split, preventing silent metric bias from skewed splits.
- 4. CustomLSTMCell == nn.LSTMCell** After copying weights, `torch.allclose` confirms < 1e-5 difference, all attributable to float32 rounding order.

Future Improvements

- 1. Bidirectional LSTM** — process sequence left-to-right and right-to-left; concatenate both final hidden states for richer classification signal.